PowerPredict

Plataforma de Previsão e Detecção de Anomalias de Consumo Energético

Documentação Técnica

Descrição · Problema · Tecnologias · Arquitetura

1. Descrição da Solução

O PowerPredict é uma aplicação inteligente voltada para monitoramento e otimização do consumo de energia em instalações prediais ou industriais. O projeto coleta dados históricos de consumo energético, processa e estrutura esses dados para análise, identifica padrões e desvios de comportamento, e utiliza aprendizado supervisionado para treinar um modelo preditivo do fator de utilização.

Com essas previsões, é possível planejar melhor o consumo de energia e identificar locais ou momentos de desperdício, promovendo eficiência operacional e sustentabilidade. Além disso, o projeto fornece métricas interpretáveis (como MAE, RMSE e R^2) e mantém um pipeline de dados robusto que permite treinar novos modelos sempre que a base é atualizada.

- Coleta dados históricos de consumo, variáveis temporais (hora, dia, mês), fatores ambientais (temperatura), características do local (feriado, fim de semana, intensidade operacional) e indicadores de anomalias.
- Treina um modelo preditivo usando aprendizado supervisionado (Random Forest) para estimar o fator de utilização da energia.

- Permite visualizar métricas e previsões, auxiliando na tomada de decisão para eficiência energética.
- Mantém um pipeline de dados que permite atualizar a base e treinar modelos continuamente.

Desafios atendidos: (1) Análise e Exploração de Dados · (2) Previsão e Forecasting · (3) Detecção de Anomalias · (9) Aprendizado Supervisionado · (15) Sustentabilidade e Eficiência

2. Problema Abordado

Muitas empresas enfrentam dificuldades em identificar desperdício de energia e padrões de consumo fora do normal. Isso pode gerar custos excessivos, falhas operacionais ou desperdício de recursos. O PowerPredict resolve este problema ao:

Desafio	Como o PowerPredict resolve
Consumo irregular e não previsto	Prevê o fator de utilização e o consumo futuro com Random Forest
Anomalias não detectadas	IsolationForest identifica automaticamente desvios anômalos no consumo
Dados históricos subutilizados	Pipeline estruturado que processa, limpa e treina modelos com os dados acumulados
Processos manuais de análise	Interface web automatiza todo o fluxo: upload → treino → previsão → análise
Falta de visibilidade operacional	Dashboards com gráficos, rankings e métricas interpretáveis (MAE, RMSE, R^2)

3. Tecnologias Utilizadas

Categoria	Tecnologias & Ferramentas
Backend	Python · Django · Django REST Framework
Banco de Dados	SQLite (desenvolvimento)
Data Science / IA	Pandas · Scikit-learn · Joblib
Modelos de ML	Random Forest Regressor (supervisionado) · IsolationForest (anomalias)
Frontend	React · JavaScript · Vite · CSS
Visualização	BarChart (SVG nativo — sem dependências externas)
Ferramentas Auxiliares	Git · Jupyter Notebook · Pipeline (experimentação de modelos)

Modelos de Machine Learning

RF	**Random Forest Regressor**

Modelo supervisionado treinado sobre registros históricos sem anomalia. Prevê o fator de utilização (consumo_kwh / potencia_kw_ac_ref). Parâmetros: n_estimators=150, max_depth=18, split treino/teste 80/20.

IF

IsolationForest

Modelo não supervisionado que detecta outliers com base na estrutura dos dados. Usa o fator_utilizacao_previsto como feature principal. Parâmetros: contamination=0.02, n_estimators=200.

4. Arquitetura da Aplicação

4.1 Visão Geral

Frontend	Backend	Dados
React + Vite	Django + DRF	SQLite
porta 5173	porta 8000	+ arquivos .pkl

4.2 Fluxo Completo da Aplicação

FASE 1 — PREPARAÇÃO DO DATASET

1

Upload do CSV

O usuário seleciona o CSV gerado pelo script_simulado.py (~150k linhas) e envia via POST /api/upload/. O backend processa em chunks de 20.000 linhas com bulk_create em batches de 5.000, salvando na tabela ConsumoEnergia.

→

Banco de Dados

Tabela ConsumoEnergia armazena: nome, tipo_local, data_hora, hora, mes, dia_semana, temperatura, consumo_kwh, potencia_kw_ac_ref, fim_de_semana, feriado, anomalia, ...

FASE 2 — TREINAMENTO DO MODELO

2

Treinamento do RandomForest

O usuário clica em Treinar Modelo via POST /api/modelo/treinar/. O sistema calcula o target (fator_utilizacao = consumo_kwh / potencia_kw_ac_ref), faz split 80/20 e treina o RandomForestRegressor com n_estimators=150, max_depth=18.

→

Artefatos salvos em disco

ml_models/modelo_consumo.pkl (modelo serializado) + ml_models/modelo_consumo_info.json (R^2, MAE, RMSE, importância das features, data do treino).

FASE 3A — PREDIÇÃO UNITÁRIA

3A

Previsão de um instante

O usuário preenche os campos (hora, temperatura, potência, etc.) e envia via POST /api/prever/. O backend carrega o modelo .pkl, aplica predict() e retorna consumo_kwh_previsto = fator_utilizacao × potencia_kw_ac_ref.

FASE 3B — ANÁLISE DE CSV + ANOMALIAS

3B

Análise em lote do CSV

O usuário sobe um novo CSV via POST /api/analisar-csv/. O RandomForest gera consumo_kwh_previsto e fator_utilizacao_previsto para cada linha. Em seguida, o IsolationForest (contamination=0.02) marca anomalia_csv=True/False.

→

Retorno consolidado

JSON com: resumo (total, anomalias, percentual, consumo médio/total), preview das primeiras 100 linhas, lista de anomalias, gráficos (por hora, por mês, por dia da semana) e CSV base64 para download.

4.3 Endpoints da API REST

Método	Endpoint	Função
POST	/api/upload/	Importa CSV histórico (chunks de 20k, bulk_create de 5k)
POST	/api/modelo/treinar/	Treina RandomForest e salva .pkl + métricas .json
GET	/api/modelo-info/	Retorna R^2, MAE, RMSE e importância das features
POST	/api/prever/	Previsão unitária de consumo via JSON
POST	/api/analisar-csv/	RandomForest + IsolationForest em lote sobre CSV
POST	/api/detectar-anomalias/	Roda IsolationForest no banco e atualiza campo anomalia
GET	/api/anomalias/	Lista registros com anomalia=True
GET	/api/consumo/	Lista registros de consumo (limite configurável)
DELETE	/api/limpar-base/	Remove todos os registros e apaga os arquivos de modelo

4.4 Responsabilidade dos Arquivos

Backend

Arquivo	O que faz
`apps/monitoramento/models.py`	Define a tabela ConsumoEnergia — uma linha = uma leitura horária
`apps/monitoramento/views.py`	Recebe requisições HTTP, valida entrada, orquestra services, devolve Response
`apps/monitoramento/urls.py`	Roteamento da app — mapeia cada endpoint para sua view
`apps/monitoramento/serializers.py`	Converte objetos Django em JSON para a API
`services/treinamento.py`	Treina o RandomForest, salva .pkl e .json de métricas
`services/previsao.py`	Previsão unitária e em lote usando o modelo salvo
`services/deteccao_anomalia.py`	IsolationForest no banco e em DataFrames externos
`services/modelo_info.py`	Lê o .json de métricas e retorna para o frontend
`services/processamento.py`	Helpers de limpeza e transformação de dados
`ml_models/modelo_consumo.pkl`	Modelo treinado serializado (gerado dinamicamente)
`ml_models/modelo_consumo_info.json`	Métricas R^2, MAE, RMSE, importância das features
`apps/monitoramento/migrations/`	Migration inicial — cria a tabela no banco de dados
`apps/monitoramento/admin.py`	Registra os models no painel Django Admin
`manage.py`	Entry point do Django — runserver, migrate, etc.
`requirements.txt`	Lista todas as dependências Python do projeto
`config/settings.py`	Configurações globais: banco, apps, CORS, timezone
`config/urls.py`	Roteamento raiz — inclui as rotas de cada app
`config/wsgi.py / asgi.py`	Interfaces para deploy em produção

Frontend

Arquivo	O que faz
`frontend/src/App.jsx`	Toda a UI: estado global, navegação, handlers e componentes
`frontend/src/services/api.js`	Funções de chamada HTTP para cada endpoint do backend
`frontend/src/App.css`	Design system dark industrial da aplicação
`frontend/src/index.css`	Reset e estilos base globais
`frontend/src/main.jsx`	Entry point React — monta o componente raiz no DOM
`frontend/index.html`	HTML raiz da SPA React
`frontend/package.json`	Dependências e scripts Node do frontend

Arquivo	O que faz
`frontend/vite.config.js`	Configuração do Vite — proxy para o backend na porta 8000

Desenvolvido com foco em modularidade, escalabilidade e usabilidade para equipes de engenharia e facilities.